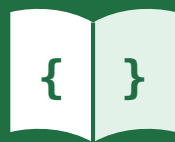

Module 07 — Spring Data MongoDB

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — A Product Catalogue with Variable Attributes (Easy)	2
Exercise 2 — Revenue per Category (Medium)	2
Exercise 3 — Weighted Full-Text Search (Hard)	3
Bonus Challenge (Optional)	3

How to Work

1. The starter code is in `modules/07-data-mongodb/exercise/`. Find the TODO comments.
2. The tests run against a real MongoDB (Testcontainers). **Docker must be running.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/07-data-mongodb/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/07-data-mongodb/solution/`.

★ Tip

The exercises are independent of each other. Every test class inserts its own data.

Exercise 1 — A Product Catalogue with Variable Attributes (Easy)

Goal: store different kinds of products in the same collection without changing a schema.

Books have a page count, bags a colour and a material, pens a colour. All of them live in the products collection.

Tasks (package `exercisel`):

- TODO 1a — Product: stored in the products collection, and no two products may share a sku.
- TODO 1b — `findByCategoryOrderByPrice`: the products of a category, cheapest first. Use a derived query.
- TODO 1c — `findByColor`: a JSON query on the color entry of the attributes map.

Hints:

- Lesson sections 3.1 and 3.5: `@Document("...")`, `@Indexed(unique = true)`.
- Lesson section 3.2: `@Query("{ 'attributes.language': ?0 })`.
- Deleting the default method and declaring an abstract method instead is enough.

Acceptance criteria: all 4 tests in `ExerciselTest` pass.

Estimated time: 20 minutes

Exercise 2 — Revenue per Category (Medium)

Goal: compute a report inside the database with an aggregation pipeline, not in the application.

Tasks (exercise2.SalesReport.revenuePerCategory):

- TODO 2a — only sales with `from <= soldAt < until`.
- TODO 2b — per category: `units = sum of quantity`, `revenue = sum of quantity * unitPrice`.
- TODO 2c — map `_id` to the category field and sort by revenue, highest first.

Hints:

- Lesson section 3.4: `newAggregation(match(...), group(...), project(...), sort(...))`.
- `where("soldAt").gte(from).lt(until)`.
- The sum of a product: `.sum(ArithmeticOperators.Multiply.valueOf("quantity").multiplyBy("unitPrice"))`.
- To map the result to a record: `mongo.aggregate(pipeline, Sale.class, CategoryRevenue.class)`.

Acceptance criteria: both tests in Exercise2Test pass.

Estimated time: 40 minutes

Exercise 3 — Weighted Full-Text Search (Hard)

Goal: a search where a word in the title counts more than the same word in the body.

Tasks (package exercise3):

- TODO 3a — Article: title (weight 3) and body (weight 1) are part of the text index.
- TODO 3b — `ArticleSearch.search`: return the articles containing **any** of the words, sorted by relevance score, at most limit of them.

Hints:

- `@TextIndexed(weight = 3)`.
- `TextQuery.queryText(TextCriteria.forDefaultLanguage().matchingAny(...)).sortByScore()` and `query.limit(...)`.
- To split the words on spaces: `words.split("\\s+").`
- Text search recognises stems: searching “indexes” finds articles containing “index”.

Acceptance criteria: all 4 tests in Exercise3Test pass.

Estimated time: 40 minutes

Bonus Challenge (Optional)

Extend the report of Exercise 2 with `$facet`: one pipeline returns both the revenue per category and the total revenue in a single query. Use `explain()` to see how an index on `soldAt` changes the query.